

Data Structures

Date: 24-10-2025

Instructor

Mr. Muhammad Naveed

TA

Menahil Amir

Assignment #3

Course code: CS2001

Total marks: 100

Questions: 4

Roll No

Section

Student Signature

CLO 02: Demonstrate basic concepts of Data Structures

Attempt all the questions.

QUESTION 1: (20 Marks)

An online multiplayer game maintains a leaderboard of players. Each player has a unique username (string) and a score (integer). Since millions of players are active, searching for a player's score must be efficient. The developers decide to use hashing with collision handling.

You are required to design and implement a hash table to store and manage player data. The hash table must use a polynomial rolling hash function for string usernames and support the following operations:

1. Insert Player Data
 - Insert (username, score) into the hash table.
 - If the username already exists, update the score.
2. Search Player Score
 - Given a username, return the player's score if found, otherwise "Not Found".
3. Delete Player
 - Remove a username from the hash table.
 - Ensure search still works correctly after deletion.
4. Process Queries
 - Given a list of usernames, output their scores or "Not Found".

Hash Function Requirement

You must implement the polynomial rolling hash function:

$$h(s) = \left(\sum_{i=0}^{n-1} (s[i] \cdot p^i) \right) \bmod m$$

- $s[i]$ = ASCII value of character at position i

National University of Computer and Emerging Sciences

Lahore Campus

- pp = prime base (e.g., 31 or 53)
- mm = large prime modulus (e.g., 10^9+9)

The final table index is computed as:

$index = h(s) \bmod TableSize$

Input Format

- First line: integer $TableSize$ (size of the hash table).
- Second line: list of player data (username, score) pairs.
- Third line: list of query usernames.

Example

Input

$TableSize = 7$

Players = [("Alice", 1200), ("Bob", 950), ("Charlie", 1500), ("David", 800), ("Eve", 1100)]

Queries = ["Charlie", "Eve", "Frank", "Alice"]

Output

Charlie -> 1500

Eve -> 1100

Frank -> Not Found

Alice -> 1200

Question 2: (20 Marks)

You are required to design and implement a program that uses Huffman Coding to compress and decompress text. The program must support the following operations:

1. Frequency Analysis
 - Count the frequency of each character in the given input string.
2. Build Huffman Tree
 - Construct a binary tree based on character frequencies.
 - Each leaf node represents a character, and the path from root to leaf represents its binary code.
3. Generate Huffman Codes
 - Assign binary codes to each character using the Huffman tree.
 - Ensure that no code is a prefix of another (prefix-free property).
4. Encode Text
 - Convert the input string into a compressed binary string using the generated Huffman codes.
5. Decode Text
 - Given the compressed binary string and the Huffman tree, reconstruct the original text.

National University of Computer and Emerging Sciences

Lahore Campus

Input Format

- First line: a string S (the text to be compressed).

Output Format

1. Print the frequency of each character.
2. Print the Huffman code assigned to each character.
3. Print the encoded binary string.
4. Print the decoded string (should match the original input).

Question 3: (30 Marks)

You are required to design and implement a program that models the city's transportation network as a weighted graph. The program must support the following operations:

1. Graph Construction
 - Build a weighted graph using adjacency lists.
 - Nodes represent intersections, edges represent roads, and weights represent travel times.
2. Shortest Path Analysis
 - Implement Dijkstra's algorithm to find the shortest travel time between two intersections.
 - Handle cases where no path exists.
3. Cycle Detection
 - Implement a function to detect if the network contains any cycles (using DFS).
 - This helps identify potential traffic loops.
4. Connectivity Check
 - Determine if the graph is connected (i.e., every intersection can reach every other).
 - If not connected, identify the number of connected components.
5. Critical Road Analysis (Bridge Detection)
 - Implement an algorithm to find bridges (edges whose removal disconnects the graph).
 - This helps identify roads critical to city connectivity.

Question 4: (30 Marks)

A fire has broken out in a multi-floor building represented as a matrix grid. Each cell in the matrix can be:

- 0 → empty space (safe to move)
- 1 → wall (cannot pass)
- F → fire (dangerous, cannot pass)
- S → starting position of the rescuer
- E → exit point

The rescuer must find a safe path from S to E using BFS or DFS traversal.

National University of Computer and Emerging Sciences

Lahore Campus

You are required to design and implement a program that:

1. Matrix Representation
 - Read a matrix of size $N \times M$ representing the building.
2. Path Search
 - Use BFS (preferred for shortest path) or DFS to determine if a path exists from S to E.
 - The rescuer can move up, down, left, right (no diagonals).
3. Shortest Path Length
 - If a path exists, output the minimum number of steps required to reach the exit.
4. Path Reconstruction
 - Print the actual path taken (sequence of coordinates).
5. Failure Case
 - If no path exists, output "No Escape".

Input Format

- First line: integers N and M (rows and columns).
- Next N lines: matrix with characters 0, 1, F, S, E.

Output Format

- If path exists: print "Escape possible in X steps" (where X is shortest path length).
- If no path exists: print "No Escape".

Example

Input

N = 5, M = 6

Matrix:

S 0 1 0 0 0

0 F 1 0 F 0

0 0 0 1 0

1 1 0 F 0 0

0 0 0 0 E

Output

Escape possible in 9 steps (and the path taken -coordinates)